

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: ГЕНЕТИЧЕСКИЕ АЛГОРИТМЫ

Студенты гр. 4343

Я.Р. Гизатов
Н.С. Стариков
И.А. Толмачев

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты: Я.Р. Гизатов

Н.С. Стариков

И.А. Толмачев

Группа: 4343

Тема практики: Генетические алгоритмы

Задание на практику:

Дан взвешенный неориентированный граф (ребра имеют вес больше 0).
Необходимо найти минимальное остовное дерево для данного графа,
используя генетические алгоритмы.

Сроки прохождения практики: 06.07.2026 – 17.07.2026

Дата сдачи отчета: 14.07.2026

Дата защиты отчета: 14.07.2026

Студенты

Я.Р. Гизатов

Н.С. Стариков

И.А. Толмачев

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

В отчете представлены результаты выполнения учебной практики, посвященной исследованию и реализации генетического алгоритма для решения задачи поиска минимального остовного дерева взвешенного неориентированного графа. Он содержит описание предметной области, постановку задачи, результаты разработки, сведения об использованных технологиях и выводы по итогам выполненной работы.

SUMMARY

This report presents the results of an academic project focused on researching and implementing a genetic algorithm to solve the problem of finding the minimum spanning tree of a weighted undirected graph. It includes a description of the problem domain, a problem statement, development results, details regarding the technologies used, and conclusions based on the work performed.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	8
1.1 Предметная область	8
1.2 Задачи практики	8
1.2.1 Изучение предметной области	8
1.2.2 Анализ требований к программному средству.....	9
1.2.3 Проектирование структуры программного решения	9
1.2.4 Проектирование генетического алгоритма.....	9
1.2.5 Разработка графического пользовательского интерфейса.....	9
1.2.6 Реализация способов задания исходных данных	9
1.2.7 Анализ результатов	10
2 РЕЗУЛЬТАТЫ РАБОТЫ	11
2.1 Распределение ролей в бригаде	11
2.2 Проектирование формата хранения графа	11
2.3 Генерация случайного графа	12
2.4 Проектирование программного решения для генетического алгоритма ..	12
2.4.1 Представление особи	13
2.4.2 Функция исправления решения.....	13
2.4.3 Функция приспособленности	14
2.4.4 Оператор отбора	14
2.4.5 Оператор скрещивания	15
2.4.6 Оператор мутации	16
2.4.7 Критерий завершения алгоритма	16
2.5 Проектирование графического интерфейса.....	16
2.5.1 Блок «Данные графа».....	17
2.5.2 Блок «Параметры генетического алгоритма».....	17
2.5.3 Блок «Информация».....	18
2.5.4 Блок «Управление»	19
2.5.5 Блок «Визуализация»	19
2.5.6 Блок «График эволюции»	21

2.6 Проектирование интеграции основного алгоритма в графический интерфейс	21
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	24
3.1 Генератор случайных чисел для генерации графа.....	24
3.2 Распределения для операторов генетического алгоритма	24
3.3 Библиотеки для графического интерфейса	25
3.3.1 Библиотека Dear ImGui	25
3.3.2 Библиотека GLFW.....	25
3.3.3 Библиотека ImPlot	26
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	27
ЗАКЛЮЧЕНИЕ	28
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30
ПРИЛОЖЕНИЕ А	31

ВВЕДЕНИЕ

Предметной областью учебной практики являются методы решения задач комбинаторной оптимизации с использованием генетических алгоритмов, которые моделируют процессы естественного отбора и наследования для поиска решений с минимальным значением целевой функции. В рамках практики рассматривается задача поиска минимального остовного дерева взвешенного неориентированного графа, позволяющая продемонстрировать применение таких алгоритмов при решении сложных оптимизационных задач.

Целью учебной практики является разработка программного средства, реализующего поиск минимального остовного дерева с использованием генетического алгоритма, а также исследование особенностей его работы и визуализация процесса поиска решения.

Для достижения поставленной цели необходимо решить следующие задачи:

- Изучить теоретические основы генетических алгоритмов и особенности их применения при решении задач комбинаторной оптимизации;
- Изучить методы поиска минимального остовного дерева и определить особенности применения генетического алгоритма для решения данной задачи;
- Разработать способ представления графа и кодирования решения в виде хромосомы;
- Определить функцию приспособленности и выбрать генетические операторы отбора, скрещивания и мутации;
- Разработать архитектуру программного средства и определить взаимодействие основных программных модулей;
- Реализовать алгоритм поиска минимального остовного дерева с использованием генетического алгоритма;

- Разработать графический пользовательский интерфейс, обеспечивающий ввод исходных данных, настройку параметров алгоритма, управление процессом вычислений и визуализацию промежуточных и конечных результатов;
- Реализовать возможность задания исходных данных посредством ручного ввода, чтения из файла и случайной генерации графа;
 - Выполнить проверку работоспособности разработанного программного средства на различных наборах данных и провести анализ полученных результатов.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Одной из важных задач теории графов является поиск минимального остовного дерева взвешенного графа. Остовным деревом называется подграф, содержащий все вершины исходного графа, не имеющий циклов и состоящий из минимально необходимого количества ребер. Минимальным остовным деревом является такое остовное дерево, суммарный вес ребер которого минимален среди всех возможных остовных деревьев данного графа.

Традиционно данная задача эффективно решается традиционными алгоритмами, такими как алгоритмы Краскала и Прима. Однако использование генетических алгоритмов позволяет рассмотреть задачу с точки зрения методов оптимизации, основанных на моделировании процессов естественного отбора, наследования и мутации.

В рамках данной практики рассматривается реализация генетического алгоритма поиска минимального остовного дерева взвешенного неориентированного графа. Алгоритм должен обеспечивать формирование начальной популяции решений, вычисление функции приспособленности, выполнение операций отбора, скрещивания и мутации, а также последовательное улучшение найденных решений до достижения критерия остановки. Дополнительно предусматривается визуализация процесса работы алгоритма и анализ изменения качества решений по поколениям.

1.2 Задачи практики

1.2.1 Изучение предметной области

Изучить особенности задачи поиска минимального остовного дерева взвешенного неориентированного графа, рассмотреть существующие алгоритмы ее решения, а также исследовать принципы работы генетических алгоритмов и возможность их применения к задачам комбинаторной оптимизации.

1.2.2 Анализ требований к программному средству

Проанализировать требования, предъявляемые к разрабатываемому программному обеспечению, определить способы задания исходных данных, параметры настройки генетического алгоритма, требования к визуализации процесса поиска решения и отображению результатов работы.

1.2.3 Проектирование структуры программного решения

Разработать архитектуру программного средства, определить основные классы, структуры данных и взаимодействие отдельных модулей, отвечающих за представление графа, выполнение генетического алгоритма и взаимодействие с графическим интерфейсом.

1.2.4 Проектирование генетического алгоритма

Разработать структуру генетического алгоритма поиска минимального остова дерева, определить способ кодирования решений, выбрать функцию приспособленности, методы отбора, скрещивания и мутации, а также установить критерий завершения работы алгоритма.

1.2.5 Разработка графического пользовательского интерфейса

Разработать графический пользовательский интерфейс, обеспечивающий взаимодействие пользователя с программой. Интерфейс должен предоставлять возможность создания и редактирования графа, загрузки данных из файла, случайной генерации входных данных, настройки параметров генетического алгоритма, запуска и пошагового выполнения вычислений вперед и назад, отображения текущего состояния алгоритма, найденного остова дерева, а также графика изменения функции качества по поколениям.

1.2.6 Реализация способов задания исходных данных

Обеспечить возможность задания исходного графа несколькими способами: ручным вводом через графический интерфейс, чтением из файла и случайной генерацией графа.

1.2.7 Анализ результатов

Проанализировать результаты работы разработанного программного средства, оценить эффективность выбранных методов и сделать выводы о возможности применения генетического алгоритма для решения задачи поиска минимального остовного дерева.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1 Распределение ролей в бригаде

Проектированием графического интерфейса занимался Я.Р. Гизатов.

Проектированием формата хранения графа, генерации случайного графа и подключением генетического алгоритма к графическому интерфейсу занимался Н.С. Стариков.

Проектированием программного решения для генетического алгоритма занимался И.А. Толмачев.

2.2 Проектирование формата хранения графа

Для хранения исходного графа выбрана структура данных, основанная на списке ребер. Такое представление является универсальным, компактным и удобно используется как при чтении графа из файла, так и при случайной генерации или вводе пользователем через графический интерфейс.

Каждое ребро графа содержит номера двух соединяемых вершин и вес ребра. В программной реализации ребро описывается структурой:

```
struct Edge
{
    int from;
    int to;
    double weight;
};
```

Полный граф представляет собой количество вершин и список всех ребер:

```
struct Graph
{
    int vertexCount;
    std::vector<Edge> edges;
};
```

Данный способ хранения позволяет эффективно получать информацию о каждом ребре и использовать его индекс при построении хромосом генетического алгоритма.

Исходные данные также могут храниться в текстовом файле. В первой строке располагается количество вершин и количество ребер, после чего перечисляются все ребра графа в формате: *u v weight*, где *u* и *v* – вершины,

соединяемые ребром, *weight* – вес ребра. Такой формат является простым для чтения и легко редактируется вручную.

2.3 Генерация случайного графа

При генерации пользователь может задать количество вершин и коэффициент полноты графа. На основании этих параметров вычисляется необходимое количество ребер, которое затем используется для построения графа. Минимальное число плотности графа $\frac{2}{n}$, а максимальное $\frac{n(n-1)}{2}$.

Построение случайного графа выполняется в два этапа. На первом этапе формируется связное остовное дерево, соединяющее все вершины графа. Для этого последовательно создаются ребра между вершинами с номерами i и $i + 1$. Каждому ребру случайным образом назначается положительный вес.

На втором этапе к полученному дереву добавляются дополнительные случайные ребра. При генерации новых ребер контролируется отсутствие петель и дублирования уже существующих соединений. Для этого используется множество пар вершин, позволяющее оперативно определить наличие ребра между двумя вершинами. Добавление продолжается до достижения требуемого количества ребер либо до исчерпания всех возможных вариантов.

Вес каждого ребра выбирается случайным образом из заранее заданного диапазона. В результате работы алгоритма формируется простой связный взвешенный неориентированный граф, пригодный для последующего поиска минимального остовного дерева генетическим алгоритмом.

2.4 Проектирование программного решения для генетического алгоритма

Были исследованы принципы работы генетических алгоритмов и возможность их применения к задачам комбинаторной оптимизации в источниках [1] и [2], чтобы сформировать представление о проектировании программного решения.

2.4.1 Представление особи

В данной работе каждая особь представляет собой список индексов ребер исходного графа без дубликатов. Таким образом, каждая хромосома определяет набор ребер, который рассматривается как кандидат на минимальное остовное дерево. Набор имеет длину $n - 1$, где n – количество вершин в графе.

Например, если граф содержит девять ребер и шесть вершин с индексами $\{0,1,2,3,4,5,6,7,8\}$, то особь $\{0,3,5,7,8\}$ означает, что предполагаемое дерево состоит именно из этих пяти ребер.

Такой способ кодирования обладает несколькими преимуществами: простотой реализации, небольшим объемом памяти, удобством выполнения генетических операторов, независимостью от конкретного расположения вершин графа.

2.4.2 Функция исправления решения

Недостатком реализованного представления особи является возможность появления недопустимых решений. После операций скрещивания или мутации выбранный набор ребер может содержать циклы, не соединять все вершины графа либо иметь дубликаты. Для устранения подобных ситуаций используется специальная процедура исправления решения.

Назначение процедуры *repair* – преобразование произвольного набора ребер в допустимое остовное дерево. Исправление выполняется в три последовательных этапа, каждый из которых устраняет свой тип нарушений:

- Устранение дубликатов. Набор ребер особи может содержать повторяющиеся индексы. Для их удаления производится сортировка списка индексов, после чего последовательные дубликаты исключаются с помощью стандартного алгоритма *unique*. В результате каждый индекс встречается не более одного раза.

- Исправление циклов. Для построения ациклического графа используется структура данных для непересекающихся множеств, инициализируемая для всех вершин графа. Ребра обрабатываются в том порядке, в котором они присутствуют в текущем наборе особи. Если добавление очередного ребра не создает цикла, ребро сохраняется в результирующий список, в противном случае оно отбрасывается. Таким образом, формируется лес, содержащий максимально возможное подмножество исходных ребер, не содержащее циклов.
- Восстановление связности. Если после предыдущего этапа количество сохраненных ребер меньше $n - 1$, где n – количество вершин, граф не является связным. Для объединения образовавшихся компонент выполняется просмотр всех ребер исходного графа в порядке возрастания их весов. Каждое ребро добавляется в дерево, если оно соединяет две разные компоненты связности. Процесс продолжается до тех пор, пока не будет набрано ровно $n - 1$ ребер, что гарантирует получение связного остова.

2.4.3 Функция приспособленности

Качество каждой особи оценивается функцией приспособленности. Поскольку целью задачи является минимизация суммарного веса остова дерева, в качестве оценки используется прямая зависимость между стоимостью дерева и его приспособленностью.

Пусть W – суммарный вес ребер дерева. Тогда функция приспособленности определяется следующим образом: $fitness = W$.

Таким образом, чем меньше общий вес найденного дерева, тем меньше значение функции приспособленности, а следовательно, тем выше вероятность участия данной особи в формировании следующего поколения.

2.4.4 Оператор отбора

Для формирования родительских пар выбран турнирный отбор. Суть метода заключается в том, что из текущей популяции случайным образом

выбирается небольшая группа особей, после чего в качестве родителя выбирается особь с наименьшим значением функции приспособленности.

Выбор данного метода обусловлен несколькими причинами:

- Простотой программной реализации;
- Отсутствием необходимости вычислять вероятности выбора каждой особи;
- Устойчивостью к различным диапазонам значений функции приспособленности. По сравнению с рулеточным отбором турнирный метод менее чувствителен к масштабу функции качества и обеспечивает стабильную работу алгоритма;
- Рациональным соотношением между исследованием новых решений и сохранением лучших найденных особей.

2.4.5 Оператор скрещивания

Для получения новых решений используется равномерное скрещивание. Для каждого гена независимо определяется, от какого родителя он будет унаследован. При формировании первого потомка для каждого гена с одинаковой вероятностью выбирается значение первого или второго родителя. Второй потомок получает противоположный выбор.

Для оператора задается вероятность выполнения, если значение меньше, то два выбранных родителя сразу отправляются в новое поколение, если значение больше, то происходит скрещивание и только их дети проходят дальше.

Основным преимуществом такого метода является возможность свободного комбинирования отдельных удачных ребер из различных решений. В отличие от одноточечного или двухточечного скрещивания отсутствует привязка к расположению генов внутри хромосомы, что особенно важно при решении задачи поиска минимального остовного дерева, где порядок расположения ребер не оказывает влияния на качество решения.

После завершения операции скрещивания выполняется процедура исправления решения.

2.4.6 Оператор мутации

Мутация предназначена для поддержания разнообразия популяции и предотвращения преждевременной сходимости алгоритма.

В разработанном алгоритме мутация выполняется следующим образом:

- Случайным образом удаляется одно ребро;
- Выполняется процедура восстановления корректного дерева.

Такой подход позволяет исследовать новые области пространства поиска без существенного нарушения структуры найденных решений.

2.4.7 Критерий завершения алгоритма

Для завершения работы генетического алгоритма используется комбинированный критерий остановки.

Алгоритм прекращает работу при выполнении одного из двух условий:

- Достигнуто максимальное количество поколений;
- Найденное решение с минимальным весом не изменяется в течение заданного количества поколений подряд.

Использование сразу двух критериев позволяет, с одной стороны, гарантировать завершение работы программы, а с другой – избежать выполнения лишних вычислений после фактической сходимости популяции.

2.5 Проектирование графического интерфейса

Для организации взаимодействия пользователя с программой был разработан графический интерфейс, позволяющий выполнять все этапы работы с генетическим алгоритмом без использования командной строки. Интерфейс предоставляет средства создания входных данных, настройки параметров алгоритма, управления процессом поиска решения и визуализации полученных результатов.

Структура графического интерфейса состоит из нескольких функциональных блоков. Полный рисунок интерфейса см. в приложении А.

2.5.1 Блок «Данные графа»

Данный блок предназначен для формирования исходного графа, на котором выполняется поиск минимального остовного дерева (см. рисунок 1).

Пользователю предоставляется возможность создания графа тремя различными способами:

- Загрузка графа из текстового файла;
- Случайная генерация графа по заданным параметрам (см. в Приложении А);
- Ручной ввод графа посредством заполнения матрицы смежности (см. в Приложении А).

После формирования графа выполняется его проверка на корректность и связность. При успешной проверке данные передаются в генетический алгоритм для последующей обработки.

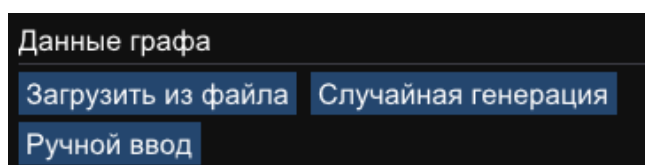


Рисунок 1 – Блок «Данные графа»

2.5.2 Блок «Параметры генетического алгоритма»

Данный блок предназначен для настройки основных параметров работы генетического алгоритма (см. рисунок 2).

Пользователь может изменять следующие параметры:

- Размер популяции;
- Размер турнирного отбора;
- Вероятность выполнения операции скрещивания;
- Вероятность выполнения мутации;
- Максимальное количество поколений;
- Максимальное число поколений без улучшения решения.

Изменение указанных параметров позволяет исследовать влияние различных настроек на скорость сходимости алгоритма и качество найденного решения.

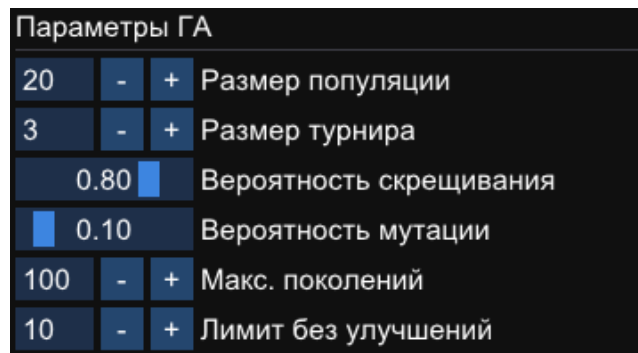


Рисунок 2 – Блок «Параметры генетического алгоритма»

2.5.3 Блок «Информация»

Блок предназначен для отображения текущего состояния алгоритма (см. рисунок 3).

- В процессе работы отображаются следующие сведения:
- Количество вершин и ребер графа;
- Номер текущего поколения;
- Размер популяции;
- Вес найденного решения с минимальным весом;
- Значение функции приспособленности;
- Текущее состояние алгоритма (ожидание, выполнение или завершение);
- Информационные сообщения о выполняемых действиях.

Наличие данного блока позволяет пользователю контролировать процесс выполнения алгоритма без необходимости анализа внутренних структур данных.

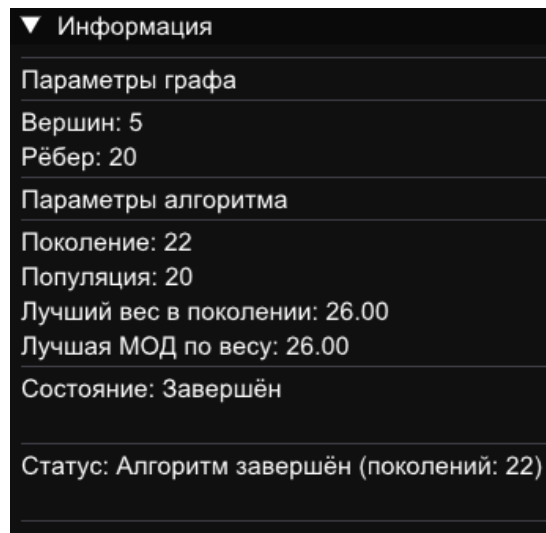


Рисунок 3 – Блок «Информация»

2.5.4 Блок «Управление»

Блок управления содержит элементы, предназначенные для запуска и пошагового выполнения генетического алгоритма (см. рисунок 4).

- Пользователю доступны следующие действия:
- Запуск алгоритма до достижения критерия остановки;
- Выполнение одного поколения алгоритма;
- Переход к предыдущему поколению;
- Полный сброс состояния алгоритма.

Пошаговое выполнение позволяет подробно проследить изменение популяции и процесса поиска решения, а возможность возврата к предыдущему поколению обеспечивает более детальный анализ работы генетического алгоритма.

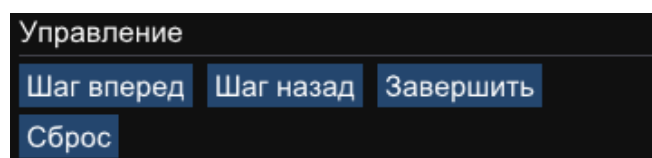


Рисунок 4 – Блок «Управление»

2.5.5 Блок «Визуализация»

Данный блок предназначен для графического отображения исследуемого графа и текущего решения (см. рисунок 5).

Все вершины и ребра исходного графа отображаются на плоскости. Пользователь может выбрать любую особь текущей популяции, после чего соответствующее ей остоное дерево выделяется цветом поверх исходного графа.

Для выбранной особи отображаются:

- Входящие в дерево ребра;
- Веса отдельных ребер;
- Суммарный вес дерева;
- Количество используемых ребер.

Таким образом обеспечивается наглядное представление структуры каждого решения, формируемого генетическим алгоритмом.

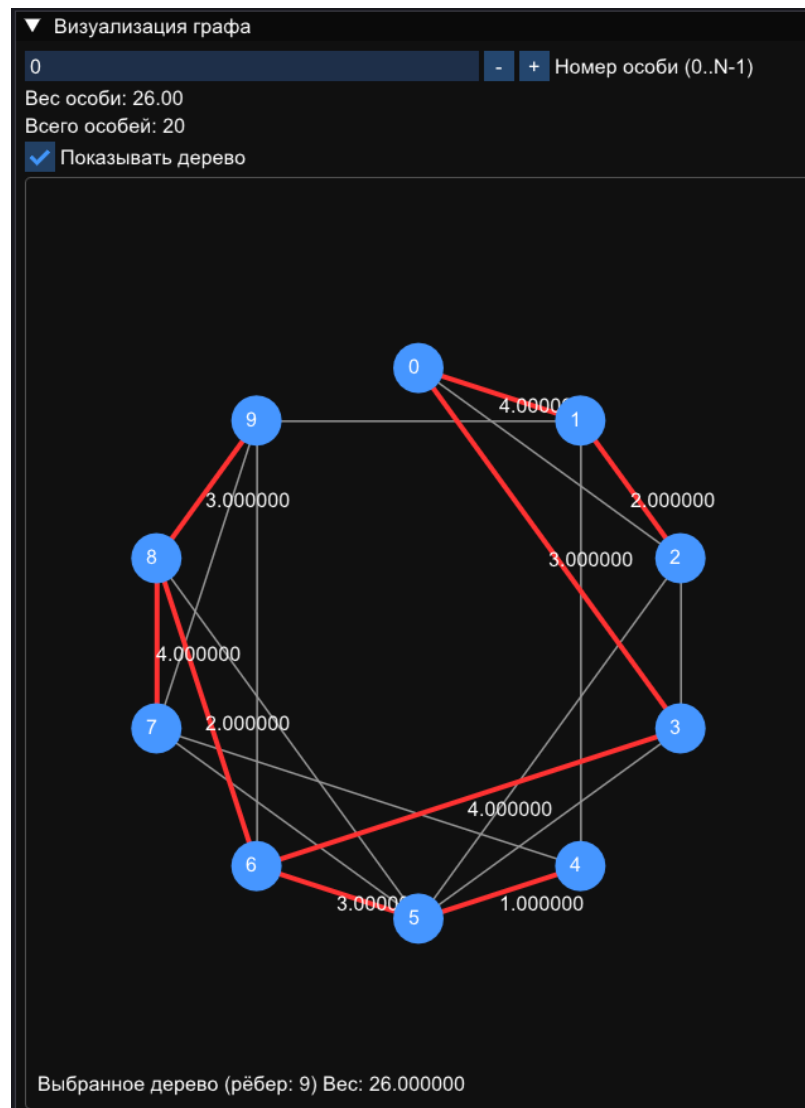


Рисунок 5 – Блок «Визуализация»

2.5.6 Блок «График эволюции»

Для анализа процесса работы алгоритма реализован график изменения качества найденного решения по поколениям (см. рисунок 6).

По горизонтальной оси откладывается номер поколения, а по вертикальной – значение функции приспособленности особи с минимальным значением функции приспособленности данного поколения.

После выполнения каждого поколения график автоматически дополняется новой точкой, что позволяет наблюдать динамику поиска решения в режиме реального времени. При переходе к предыдущему поколению отображаемый график также корректно восстанавливается в соответствии с сохраненной историей выполнения алгоритма.

Использование данного блока позволяет оценить скорость сходимости генетического алгоритма и определить момент стабилизации процесса поиска решения с минимальным значением функции приспособленности.

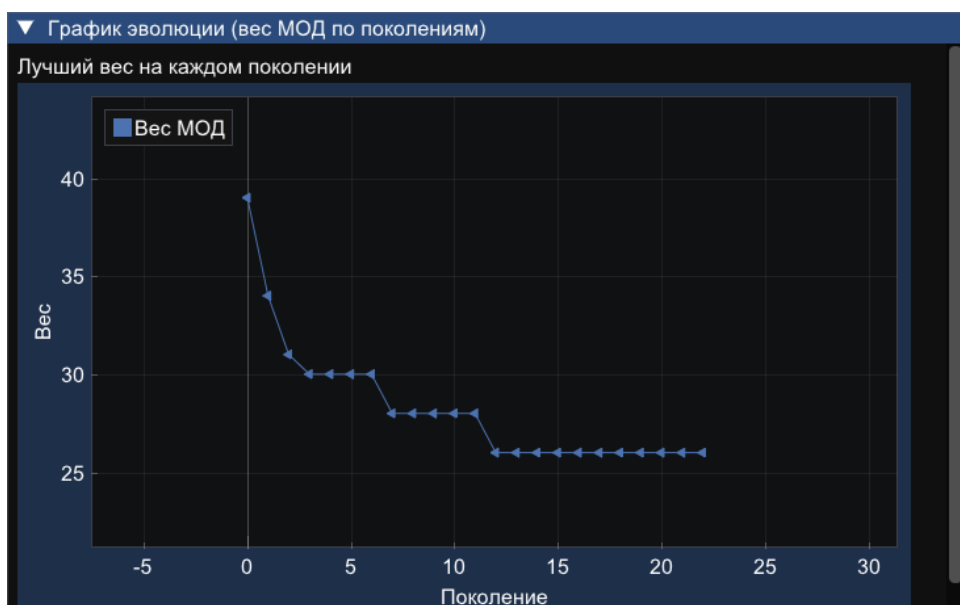


Рисунок 6 – Блок «График эволюции»

2.6 Проектирование интеграции основного алгоритма в графический интерфейс

Интеграция основного алгоритма в графический интерфейс реализована через разделение ответственности между модулями и организацию взаимодействия на основе глобальных переменных состояния. Основной

генетический алгоритм инкапсулирован в классе *GeneticAlgorithm*, который предоставляет интерфейс для установки графа и параметров, инициализации популяции, выполнения одного поколения и получения текущих результатов. Графический интерфейс, содержит отдельные панели для управления, отображения информации, визуализации графа и построения графиков эволюции.

Для синхронизации данных между интерфейсом и алгоритмом используется глобальный объект класса *GeneticAlgorithm*, доступный из любого модуля, что позволяет обработчикам кнопок и функциям отрисовки напрямую вызывать методы алгоритма и обновлять отображаемые значения.

При загрузке графа из файла, случайной генерации или ручном вводе производится инициализация алгоритма: объекту передается текущий граф, параметры, заданные пользователем (размер популяции, размер турнира, вероятности скрещивания и мутации, максимальное число поколений), после чего вызывается метод *initialize()*, формирующий начальную популяцию. С этого момента в глобальных переменных сохраняются номер поколения, минимальный вес и приспособленность, история минимальных значений для графика и набор ребер для подсветки.

Управление выполнением реализовано через кнопки: «Завершить» запускает цикл, в котором последовательно выполняются шаги до достижения критерия остановки. После завершения все результаты обновляются в интерфейсе, включая график сходимости, который строится на основе вектора истории весов; «Шаг вперед» выполняет один шаг алгоритма и отрисовывает его состояние во всех информационных полях; «Шаг назад» выполняет откат текущего решения на одно состояние назад; «Сброс» появляется после завершения основного цикла, отвечает за очистку информационных полей на состояние инициализации алгоритма для исходного графа.

Для визуализации дерева с минимальным суммарным весом используется массив индексов ребер, который передается в функцию

отрисовки графа и отображается поверх основного графа жирными красными линиями.

Дополнительно предусмотрена возможность переключения между особями текущей популяции: при изменении номера особи в соответствующем поле ввода происходит обновление подсвечиваемого дерева и вывод информации о весе выбранной особи. Таким образом, интеграция обеспечивает полный цикл работы: от ввода данных до получения и визуализации результата, а также предоставляет пользователю средства контроля и анализа процесса оптимизации.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

3.1 Генератор случайных чисел для генерации графа

В качестве основного генератора псевдослучайных чисел применялся алгоритм Mersenne Twister, реализованный классом *std::mt19937*.

Начальная инициализация генератора выполнялась с использованием объекта *std::random_device*, что обеспечивает получение различных последовательностей случайных чисел при каждом запуске программы.

Для формирования весов ребер, выбора количества вершин и других целочисленных параметров использовалось равномерное распределение *std::uniform_int_distribution*. Данное распределение обеспечивает одинаковую вероятность появления каждого значения в заданном диапазоне, благодаря чему исключается смещение в сторону отдельных весов или структур графа.

3.2 Распределения для операторов генетического алгоритма

Для принятия решений о выполнении операций скрещивания и мутации применялось распределение Бернулли *std::bernoulli_distribution*. Данное распределение позволяет получить одно из двух возможных состояний – выполнение операции или ее отсутствие – с заранее заданной вероятностью. Вероятность скрещивания определяется параметром алгоритма *crossoverProbability*, а вероятность мутации – параметром *mutationProbability*.

При реализации равномерного скрещивания дополнительно использовалось распределение Бернулли с вероятностью 0,5. Для каждого гена независимо определяется, от какого из родителей он будет унаследован потомком. Благодаря этому оба родителя имеют равные шансы передать свои признаки каждому потомку, что способствует сохранению генетического разнообразия популяции.

Для реализации турнирного отбора применялось равномерное целочисленное распределение *std::uniform_int_distribution<int>*. С его помощью случайным образом выбираются особи из текущей популяции, которые затем участвуют в турнире. Каждая особь имеет одинаковую

вероятность быть выбранной в качестве кандидата, что обеспечивает корректную реализацию механизма естественного отбора.

3.3 Библиотеки для графического интерфейса

Для разработки графического интерфейса программного комплекса использовались современные библиотеки Dear ImGui^[3], GLFW^[4] и ImPlot. Их совместное применение позволило реализовать графический интерфейс пользователя, обеспечить обработку событий, визуализацию результатов работы алгоритма и построение графиков в режиме реального времени.

3.3.1 Библиотека Dear ImGui

Основой пользовательского интерфейса стала библиотека Dear ImGui^[3], реализующая концепцию непосредственного графического интерфейса.

Данная библиотека использовалась для создания всех окон приложения, кнопок управления, полей ввода, выпадающих списков, ползунков настройки параметров генетического алгоритма и информационных панелей.

К преимуществам Dear ImGui относятся простота интеграции с приложениями на языке C++, высокая скорость работы, возможность создания прототипов интерфейсов и последующего обновления элементов интерфейса в процессе выполнения программы.

3.3.2 Библиотека GLFW

Для создания окна приложения и взаимодействия с операционной системой использовалась библиотека GLFW^[4].

Она обеспечивает:

- Создание главного окна приложения;
- Создание контекста OpenGL;
- Обработку событий клавиатуры и мыши;
- Управление главным циклом программы;
- Поддержку различных операционных систем.

Использование GLFW позволило реализовать платформонезависимое окно приложения и обеспечить корректную работу графического интерфейса.

3.3.3 Библиотека ImPlot

Для отображения процесса работы генетического алгоритма использовалась библиотека ImPlot, являющаяся расширением библиотеки Dear ImGui.

С ее помощью реализовано построение графика изменения функции приспособленности по поколениям. После выполнения каждого поколения график автоматически дополняется новой точкой, что позволяет пользователю наблюдать процесс поиска решения в режиме реального времени. При возврате к предыдущему поколению отображаемый график также корректно изменяется в соответствии с сохраненной историей выполнения алгоритма.

Использование ImPlot позволило реализовать наглядную визуализацию процесса эволюции популяции без необходимости применения специализированных библиотек для научной графики.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики была разработана программная система для поиска минимального остовного дерева взвешенного неориентированного графа с использованием генетического алгоритма. Разработанное приложение обеспечивает полный цикл работы с задачей: создание входных данных различными способами, настройку параметров алгоритма, пошаговое и автоматическое выполнение поиска решения, а также визуализацию процесса эволюции популяции и полученных результатов.

В процессе выполнения практики были изучены принципы построения генетических алгоритмов, рассмотрены основные генетические операторы, реализованы механизмы отбора, скрещивания, мутации и восстановления корректности решений. Для оценки качества особей использовалась функция приспособленности, основанная на суммарном весе остовного дерева, что позволило находить решения с малым значением целевой функции.

Была реализована возможность случайной генерации связных графов, ручного ввода данных и чтения графов из файлов. Для удобства исследования работы алгоритма разработан графический интерфейс, позволяющий изменять параметры генетического алгоритма, выполнять пошаговую визуализацию, просматривать популяции различных поколений, анализировать найденные решения и отслеживать изменение функции приспособленности на протяжении всего процесса эволюции.

В результате выполнения практики были приобретены практические навыки проектирования программных систем, реализации алгоритмов оптимизации, разработки графических интерфейсов на языке C++, а также командной разработки программного обеспечения.

Работа над проектом была распределена между участниками следующим образом.

Я.Р. Гизатов выполнил проектирование и разработку графического интерфейса приложения. Были реализованы основные окна программы, элементы управления параметрами генетического алгоритма, средства

загрузки и создания графов, визуализация найденных остовных деревьев, отображение информации о ходе выполнения алгоритма, а также график изменения функции приспособленности по поколениям.

Н.С. Стариков выполнил проектирование формата хранения графов, разработал алгоритмы случайной генерации связных графов, реализовал механизмы чтения графов из файлов и ручного ввода данных. Кроме того, была выполнена интеграция программной реализации генетического алгоритма с графическим интерфейсом и обеспечено взаимодействие между вычислительной частью программы и средствами визуализации.

И.А. Толмачев выполнил проектирование и программную реализацию генетического алгоритма поиска минимального остовного дерева. Были реализованы структуры хранения популяции, алгоритм формирования начальной популяции, турнирный отбор, равномерное скрещивание, оператор мутации, процедура восстановления корректности особей, механизмы оценки приспособленности, критерии остановки алгоритма и сохранение истории поколений для поддержки пошаговой визуализации и возврата к предыдущим состояниям.

Поставленные в рамках учебной практики цели были достигнуты, а все запланированные задачи успешно выполнены. Разработанный программный комплекс соответствует требованиям задания и демонстрирует возможность применения генетических алгоритмов для решения задач комбинаторной оптимизации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Генетический алгоритм. Просто о сложном. Рассказ Марка Андреева / Хабр [Электронный ресурс]. URL: <https://habr.com/ru/articles/128704> (дата обращения: 06.07.2026).
2. Т.В. Панченко Генетические алгоритмы под редакцией Ю.Ю. Тарасевича / Т.В. Панченко, Ю.Ю. Тарасевич // Астрахань : Издательский дом «Астраханский университет». – 2007.
3. Dear ImGui [Электронный ресурс]. URL: <https://www.dearimgui.com> (дата обращения: 06.07.2026).
4. An OpenGL library | GLFW [Электронный ресурс]. URL: <https://www.glfw.org> (дата обращения: 06.07.2026).

ПРИЛОЖЕНИЕ А

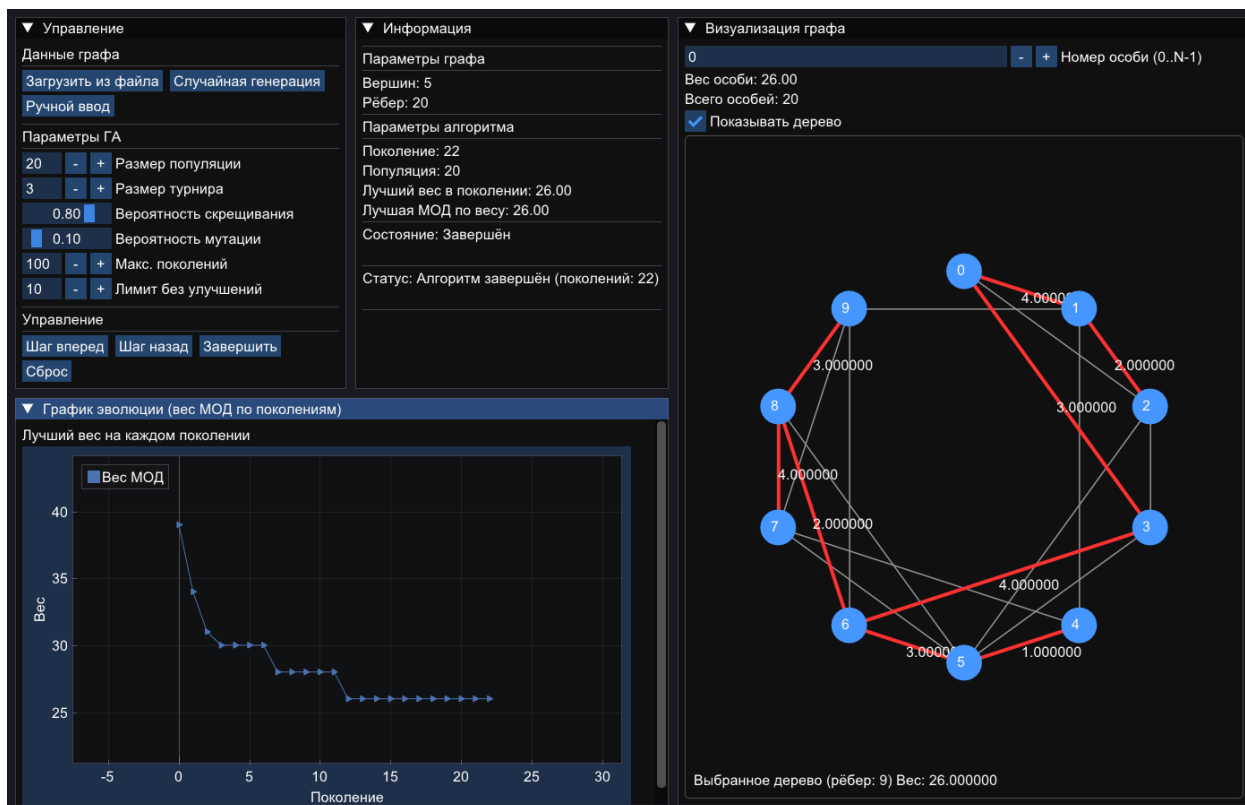


Рисунок А1 – Графический интерфейс программы

The screenshot shows the 'Ручной ввод графа' (Manual input of graph) window. It features a slider for 'Количество вершин (3-10)' (Number of vertices (3-10)) set to 5. Below the slider is a 5x5 matrix for defining edge weights between vertices 0 and 4. The matrix is as follows:

	0	1	2	3	4
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

At the bottom of the window are buttons for 'Создать граф' (Create graph) and 'Отмена' (Cancel).

Рисунок А2 – Ручной ввод графа

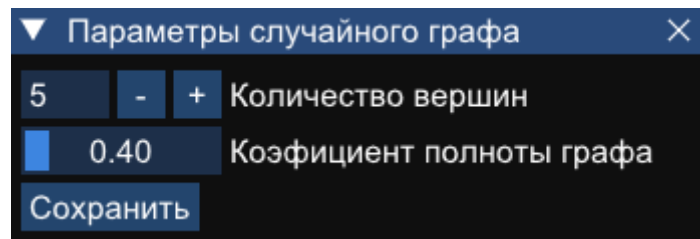


Рисунок А3 – Случайная генерация графа по заданным параметрам